第 1 题 - 流程图判断

函数 Print_values(a, b, c) 按照题目流程图中的判断结构，对 a、b、c 三个数进行比较，并按从大到小的顺序输出。这里使用嵌套 if-else 语句实现。

代表性运行结果

```
In [2]: %runfile D:/1111/ESE5023_Assignments_12642025/
PS1_1.py --wdir
a = 39
b = 98
c = 96
Output:
98 96 39
```

运行结果符合题目要求，因为 98> 96 > 39，程序能够正确按降序输出三个数。

第 2 题 - 矩阵乘法

M1 为 5 x 10 矩阵，M2 为 10 x 5 矩阵，每个元素都是 0 到 50 之间的随机整数。矩阵乘法通过三层 for 循环实现。对于结果矩阵中的每个元素 C[i][j]，程序依次计算 M1[i][k] * M2[k][j]，并对 k = 0, ..., 9 的结果进行累加。

代表性矩阵与乘积结果

```
M1:
[23, 45, 26, 40, 17, 44, 34, 32, 45, 4]
[14, 11, 24, 21, 34, 41, 19, 6, 18, 32]
[50, 36, 35, 23, 25, 34, 11, 39, 50, 17]
[31, 26, 49, 36, 38, 44, 22, 39, 2, 0]
[30, 22, 4, 32, 25, 17, 43, 45, 17, 50]
M2:
[11, 26, 31, 38, 3]
[24, 4, 18, 38, 18]
[2, 19, 49, 35, 32]
[38, 26, 35, 18, 14]
[40, 0, 16, 7, 47]
[39, 33, 5, 11, 17]
[39, 19, 9, 24, 25]
[45, 2, 46, 2, 18]
[48, 32, 48, 10, 5]
[4, 43, 42, 8, 26]
M1 * M2:
[10243, 6086, 8795, 6179, 5573]
[6226, 5088, 5947, 3761, 5102]
[9336, 6447, 10295, 6434, 5662]
[8376, 4789, 8006, 5905, 6429]
[8463, 5938, 8500, 4746, 5796]
```

计算得到的结果矩阵维度为 5 x 5，与 (5 x 10)(10 x 5) 的矩阵乘法维度关系一致。

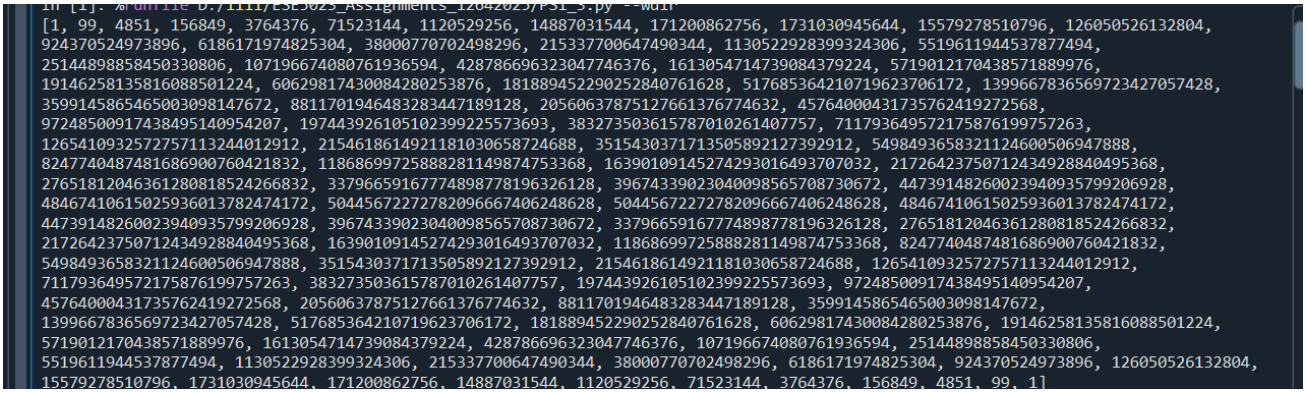
第 3 题 - 杨辉三角

Pascal_triangle(k) 从第一行 [1] 开始逐行生成杨辉三角。每一行的首尾元素均为 1，中间元素由上一行相邻两个元素之和得到。因此，该算法只需要使用列表操作、for 循环、索引和加法即可完成。

运行结果

Pascal_triangle(100) 输出 100 个整数，Pascal_triangle(20) 输出 200 个整数。可以看到每一行从两端向中间呈对称分布，符合杨辉三角的性质。

这里只展示 100 个整数



第 4 题 - 加 1 或乘 2

为了得到最少操作次数，本题从目标值 x 反向推回 1。若 x 为偶数，则可以将其除以 2，对应正向过程中的“乘 2”；若 x 为大于 1 的奇数，则先执行 x - 1，对应正向过程中的“加 1”。反向操作的步数与正向达到目标值所需的最少步数相同。

代表性运行结果

```
In [5]: %runfile D:/1111/ESE5023_Assignments_12642025/
PS1_4.py --wdir
x = 37
7
```

```
In [6]: %runfile D:/1111/ESE5023_Assignments_12642025/
PS1_4.py --wdir
x = 5
3
```

第 5 题 - 表达式穷举搜索

数字 1 到 9 之间共有 8 个空位，每个空位都有三种状态：直接连接数字、插入 +、插入 -。因此全部可能表达式的数量为 3^8 = 6,561。由于搜索空间较小，本题采用穷举法。程序将 0 到 3^8 - 1 的整数按三进制理解，从而对应每一种运算符组合。

使用 Find_expression(50) 验证

穷举搜索得到结果为 50 的有效表达式共 17 个。

```
x = 50
1-2+34-5-67+89 = 50
1+2+3+4-56+7+89 = 50
1-2-34-5-6+7+89 = 50
1+2-34+5-6-7+89 = 50
12+3+4-56+78+9 = 50
1-2+3-45+6+78+9 = 50
1-23-4-5-6+78+9 = 50
1+2+34-5-6+7+8+9 = 50
1-2-3+4+56-7-8+9 = 50
1+2+34-56+78-9 = 50
1-23+4+5-6+78-9 = 50
1-2-3-4-5-6+78-9 = 50
1-2+34+5+6+7+8-9 = 50
1+2+3-4+56-7+8-9 = 50
12-3-4-5+67-8-9 = 50
1+2-3+4+56+7-8-9 = 50
12-3+45+6+7-8-9 = 50
Out[9]: 17
```

6.2 统计 1-100 的 Total_solutions

程序将 6,561 个表达式各计算一次。若某个表达式的结果位于 1 到 100 之间，则将 Total_solutions 中对应位置

的计数加 1。这样可以一次穷举完成 1-100 所有目标值的统计，而不需要对每个目标值重复完整搜索。图 1. 目标整数 1-100 对应的有效表达式数量。统计结果表明，有效表达式数量的最大值为 26，对应目标整数 1 和 45；最小值为 6，对应目标整数 88。

